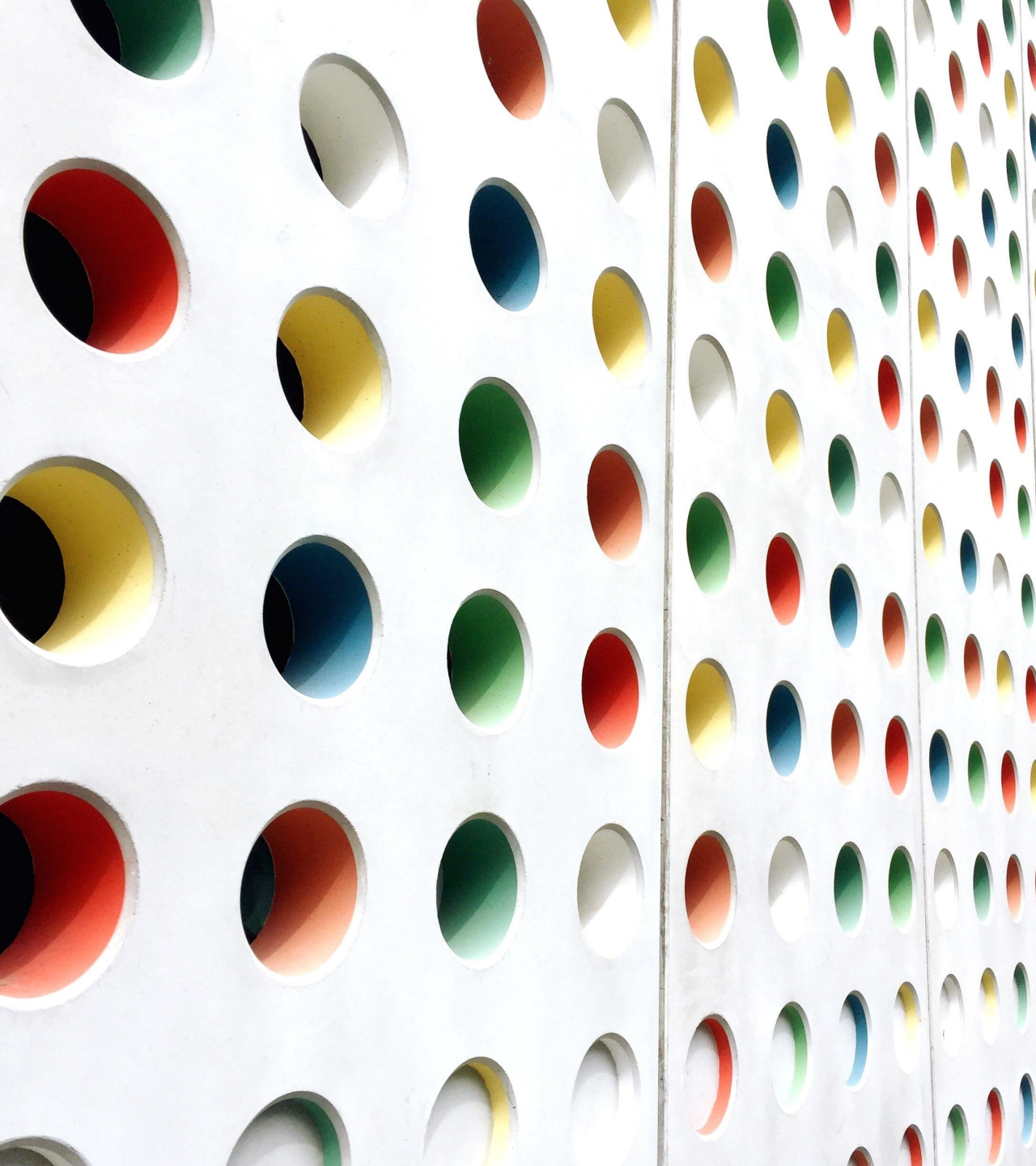




XML, XML Schema, DTD

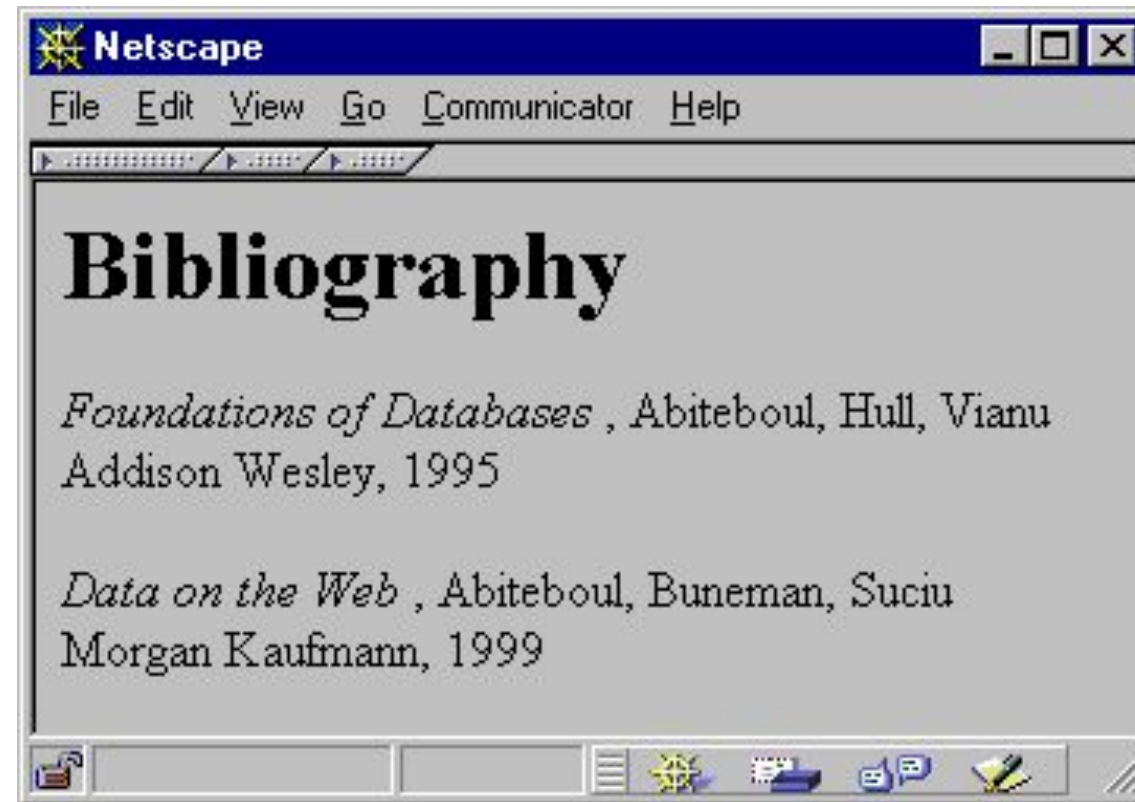
BY

SANA RIZWAN

XML— Extensible Markup Language

- XML as inheriting from the Web history
 - SGML, HTML, XHTML, XML
- XML key concepts
 - Documents, elements, attributes, text
 - Order, nested structure, textual information
- Namespaces
- XML usage scenarios
 - Financial, medical, Web Services, blogs, etc
 - communication data, meta data, documents
- DTDs and the need for describing the “structure” of an XML file

From HTML to XML



HTML describes the presentation

HTML

<h1> Bibliography </h1>

<p> <i> Foundations of Databases </i>

Abiteboul, Hull, Vianu

 Addison Wesley, 1995

<p> <i> Data on the Web </i>

Abiteboul, Buneman, Suciu

 Morgan Kaufmann, 1999

XML

<bibliography>

<book> <title> Foundations... </title>

<author> Abiteboul </author>

<author> Hull </author>

<author> Vianu </author>

<publisher> Addison Wesley </publisher>

<year> 1995 </year>

</book>

...

</bibliography>

XML describes the content

What is mark up ?

1. Literally : Notations or Symbols that are used to correct in making up text and indicate how text should be displayed.
($\lt$, $\gt$, $\wedge$, $\text{\textbackslash}$, $\text{\textasciitilde}$ etc.).
2. Mark-up refers to the sequence of Characters or other Symbols that can be inserted at certain places in a text file, to indicate how a file should look when it is printed or displayed.

What is Extensible ?

1. Literally : Capable of being Stretched out or extended.
2. In Information Technology, extensible describes a program or programming language that is designed so that users and developers can expand or add to its capabilities.

XML

A Computer language which uses **mark up** (< , > , / , ? , and **Character**) and is capable of being extended

What you need?

Note Pad, Word Pad,
Text Applications that support XML

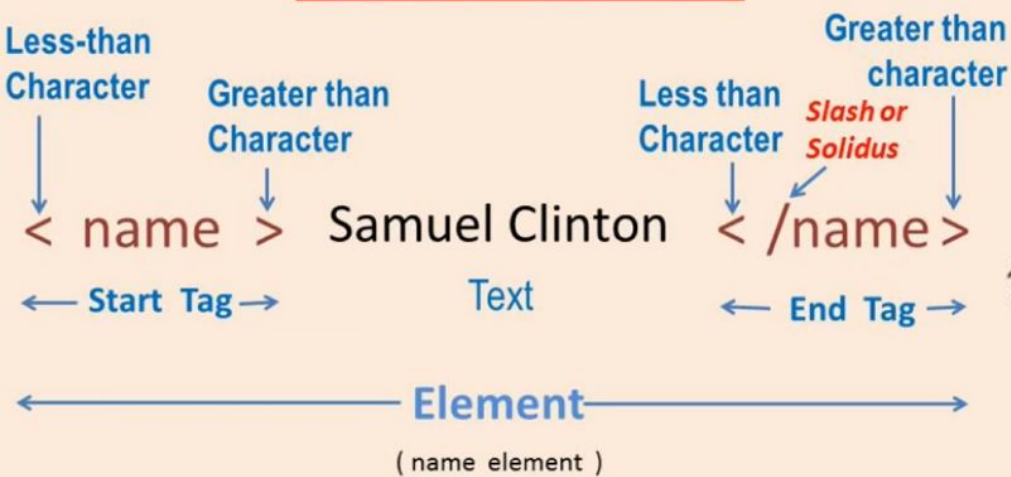
XML Editors:

XML NotePad, NotePad ++, Eclipse,
etc.

```
<?xml version="1.0" encoding="UTF-8"?>
<contact-information>
<name> Samuel Clinton </name>
<designation> Designer </designation>
<address> 123 Baker Street </address>
<phone> (123) 456-7890 </phone>
</contact-information>
```

Information in Red = Markup : Black= Ordinary Text

Tags and Elements:



Tags and Elements:

1. XML element **names** are **Case Sensitive** :

< Name > Samuel Clinton < / name > **Wrong**
< name > Samuel Clinton < / Name > **Wrong**
< name > Samuel Clinton < / name > **Correct**

2. XML document should have **One Root Element** :

< x > < / x > } **Wrong** because
< y > < / y > } No Root Element

Root Element

< a > }
< x > < / x > } **Correct**
< y > < / y > }

Structure of Elements in XML Documents

< root > can be called Parent Element

< child >

< subchild > < / subchild >

< / child >

< / root >

3. Elements **should not overlap** :

< book > < author > Robert Lang < /book > < /author > **Wrong** (Not properly nested)
< book > < author > Robert Lang < /author > < /book > **Correct** (properly nested)

XML: Attributes

```
<book price = "55" currency = "USD">  
  <title> Foundations of Databases </title>  
  <author> Abiteboul </author>  
  ...  
  <year> 1995 </year>  
</book>
```

attributes are alternative ways to represent data

More XML: Oids and References

<person id="o555"> <name> Jane </name> </person>

<person id="o456"> <name> Mary </name>

<children idref="o123 o555"/>

</person>

<person id="o123" mother="o456"><name>John</name>

</person>

oids and references in XML are just syntax

Declaration (Optional)

< ? xml version = "1.0" ? >

(indicates the number of version of xml in use)

or

< ? xml version = "1.0" encoding = "UTF-8" ? >

(indicates the number of version of xml plus character encoding used in document)

UTF stands for UCS Transformation Format ...

UCS means Universal Character SET.

Attributes

An Attribute specifies a single property for an element. It consists of a name and a value separated by an equals sign. Example...

<book xyz = "a"> <book xyz = "a">

book element ---attribute----

attribute attribute
name value

Attribute is placed within the start tag only, after the name of the element, after a white space --- no comma, semicolon etc.

Attribute names – not within quotation marks (case sensitive),

Attribute Values – Always within quotation marks, Single or Double.

Only one value for the same attribute in the same start tag.

Attributes

< book condition = old > **Wrong,**

< book "condition" = old > **Wrong,**

< / book condition = "old" > **Wrong,**

< book condition = "old" condition = "new" > **Wrong,**

< book condition = "old" > **Correct,**

< book condition = "old" some = "new" > **Correct.**

Declaration (Optional)

XML declaration is optional, but if it appears, it must be **at the Top**. Not even white space or comment should come before it

All XML parsers are required to support "UTF-8" and "UTF – 16" encodings.

XML declaration is **Case Sensitive**. It should not begin with <?XML.....

```
ExampleTwoD...  book.xml  NewFile.xml  Family.xml  Sample1.xml  17
1 <?xml version="1.0" encoding="UTF-8"?>
2 <family>
3   <father age="56"> Robert Preston </father>
4   <mother age="52"> Gale Preston </mother>
5   <firstson age="31"> Jonny
6     <grandson age="2"> Bobby </grandson>
7   </firstson>
8   <secondson age="28"> Tommy </secondson>
9   <daughter age="25"> Poly </daughter>
10 </family>
```


So Far

Differences between “xml data” versus “relational data” ?

- Data model?
- Typed?
- Homogeneity?
- Correctness?
- Usage/Purpose ?

All XML elements can be accessed through the XML DOM.

Books.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>

  <book category="cooking">
    <title lang="en">Everyday Italian</title>
    <author>Giada De Laurentiis</author>
    <year>2005</year>
    <price>30.00</price>
  </book>

  <book category="children">
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <year>2005</year>
    <price>29.99</price>
  </book>

</bookstore>
```

This code retrieves the text value of the first <title> element in an XML document:

Example

```
txt = xmlDoc.getElementsByTagName("title")[0].childNodes[0].nodeValue;
```

"XML Data Model"

Numerous competing models:

- Document Object Model (DOM):
 - class hierarchy (**node, element, attribute,...**)
 - defines API to inspect/modify the document

(The XML DOM defines a standard way for accessing and manipulating XML documents. It presents an XML document as a tree-structure.)

- XML query data model (formal)

(The XML Query Data Model defines formally the information contained in the input to an XML Query processor; in other words, an XML Query processor evaluates a query on an instance of the XML Query Data Model.)

The XML DOM is a standard for how to get, change, add, and delete XML elements.

This example loads a text string into an XML DOM object, and extracts the info from it with JavaScript:

XML query data model (formal)

XQuery and XPath Data Model, which is the data model of [XML Path Language (XPath)], [XSL Transformations (XSLT)], and [XQuery: An XML Query Language] .

The XQuery and XPath Data Model (henceforth "data model") serves two purposes.

- First, it defines the information contained in the input to an XSLT or XQuery processor.
- Second, it defines all permissible values of expressions in the XSLT, XQuery, and XPath languages.

A language is *closed* with respect to a data model if the value of every expression in the language is guaranteed to be in the data model. XSLT , XQuery , and XPath are all closed with respect to the data model.

Example

```
<html>
<body>

<p id="demo"></p>

<script>
var text, parser, xmlDoc;

text = "<bookstore><book>" +
"<title>Everyday Italian</title>" +
"<author>Giada De Laurentiis</author>" +
"<year>2005</year>" +
"</book></bookstore>";

parser = new DOMParser();
xmlDoc = parser.parseFromString(text,"text/xml");

document.getElementById("demo").innerHTML =
xmlDoc.getElementsByTagName("title")[0].childNodes[0].nodeValue;
</script>

</body>
</html>
```

XML Namespaces

<http://www.w3.org/TR/REC-xml-names>

name ::= [prefix:]localpart

```
<book xmlns:isbn="www.isbn-org.org/def">  
  <title> ... </title>  
  <number> 15 </number>  
  <isbn:number> .... </isbn:number>  
</book>
```


XML Namespaces

syntactic: `<number>` , `<isbn:number>`

semantic: provide URL for “shared” schema

```
<tag xmlns:mystyle = “http://...”>
```

...

```
<mystyle:title> ... </mystyle:title>
```

```
<mystyle:number> ...
```

```
</tag>
```

defined here

XML Namespaces provide a method to avoid element name conflicts.

Name Conflicts

In XML, element names are defined by the developer. This often results in a conflict when trying to mix XML documents from different XML applications.

This XML carries HTML table information:

```
<table>
  <tr>
    <td>Apples</td>
    <td>Bananas</td>
  </tr>
</table>
```

This XML carries information about a table (a piece of furniture):

```
<table>
  <name>African Coffee Table</name>
  <width>80</width>
  <length>120</length>
</table>
```

If these XML fragments were added together, there would be a name conflict. Both contain a `<table>` element, but the elements have different content and meaning.

XML Namespaces

Solving the Name Conflict Using a Prefix

Name conflicts in XML can easily be avoided using a name prefix.

This XML carries information about an HTML table, and a piece of furniture:

```
<h:table>
  <h:tr>
    <h:td>Apples</h:td>
    <h:td>Bananas</h:td>
  </h:tr>
</h:table>

<f:table>
  <f:name>African Coffee Table</f:name>
  <f:width>80</f:width>
  <f:length>120</f:length>
</f:table>
```

In the example above, there will be no conflict because the two `<table>` elements have different names.

XML Namespaces - The xmlns Attribute

When using prefixes in XML, a **namespace** for the prefix must be defined.

The namespace can be defined by an **xmlns** attribute in the start tag of an element.

The namespace declaration has the following syntax. `xmlns:prefix="URI"`.

```
<root>

<h:table xmlns:h="http://www.w3.org/TR/html4/">
  <h:tr>
    <h:td>Apples</h:td>
    <h:td>Bananas</h:td>
  </h:tr>
</h:table>

<f:table xmlns:f="https://www.w3schools.com/furniture">
  <f:name>African Coffee Table</f:name>
  <f:width>80</f:width>
  <f:length>120</f:length>
</f:table>

</root>
```

The xmlns attribute in the first <table> element gives the h: prefix a qualified namespace.

The xmlns attribute in the second <table> element gives the f: prefix a qualified namespace.

When a namespace is defined for an element, all child elements with the same prefix are associated with the same namespace

Namespaces can also be declared in the XML root element:

```
<root xmlns:h="http://www.w3.org/TR/html4/"
xmlns:f="https://www.w3schools.com/furniture">

  <h:table>
    <h:tr>
      <h:td>Apples</h:td>
      <h:td>Bananas</h:td>
    </h:tr>
  </h:table>

  <f:table>
    <f:name>African Coffee Table</f:name>
    <f:width>80</f:width>
    <f:length>120</f:length>
  </f:table>

</root>
```

The purpose of using an URI is to give the namespace a unique name.

However, companies often use the namespace as a pointer to a web page containing namespace information.

Uniform Resource Identifier (URI)

A **Uniform Resource Identifier** (URI) is a string of characters which identifies an Internet Resource.

The most common URI is the **Uniform Resource Locator** (URL) which identifies an Internet domain address. Another, not so common type of URI is the **Uniform Resource Name** (URN).

Default Namespace

```
xmlns="namespaceURI"
```

This XML carries HTML table information:

```
<table xmlns="http://www.w3.org/TR/html4/">
  <tr>
    <td>Apples</td>
    <td>Bananas</td>
  </tr>
</table>
```

This XML carries information about a piece of furniture:

```
<table xmlns="https://www.w3schools.com/furniture">
  <name>African Coffee Table</name>
  <width>80</width>
  <length>120</length>
</table>
```

XML DTD

What is a DTD?

DTD stands for Document Type Definition.

A DTD defines the structure and the legal elements and attributes of an XML document.

An XML document with correct syntax is called "Well Formed".

An XML document validated against a DTD is both "Well Formed" and "Valid".

Valid XML Documents

A "Valid" XML document is "Well Formed", as well as it conforms to the rules of a DTD:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE note SYSTEM "Note.dtd">
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

The DOCTYPE declaration above contains a reference to a DTD file. The content of the DTD file is shown

The DTD above is interpreted like this:

- !DOCTYPE note - Defines that the root element of the document is note
- !ELEMENT note - Defines that the note element must contain the elements: "to, from, heading, body"
- !ELEMENT to - Defines the to element to be of type "#PCDATA"
- !ELEMENT from - Defines the from element to be of type "#PCDATA"
- !ELEMENT heading - Defines the heading element to be of type "#PCDATA"
- !ELEMENT body - Defines the body element to be of type "#PCDATA"

XML DTD

The purpose of a DTD is to define the structure and the legal elements and attributes of an XML document:

Note.dtd:

```
<!DOCTYPE note
[
  <!ELEMENT note (to,from,heading,body)>
  <!ELEMENT to (#PCDATA)>
  <!ELEMENT from (#PCDATA)>
  <!ELEMENT heading (#PCDATA)>
  <!ELEMENT body (#PCDATA)>
]>
```

Task to Do

1. Make XML file “information about you” with proper tags
2. Identify NS in your XML file
3. Describe and apply DTD on XML file

XML Schema

- An XML Schema describes the structure of an XML document, just like a DTD.
- An XML document with correct syntax is called "Well Formed".
- An XML document validated against an XML Schema is both "Well Formed" and "Valid".

XML Schemas are More Powerful than DTD

- XML Schemas are written in XML
- XML Schemas are extensible to additions
- XML Schemas support data types
- XML Schemas support namespaces

Why Use an XML Schema?

With XML Schema, your XML files can carry a description of its own format.

With XML Schema, independent groups of people can agree on a standard for interchanging data. _____

With XML Schema, you can verify data.

XML Schemas Support Data Types

One of the greatest strengths of XML Schemas is the support for data types:

- It is easier to describe document content
- It is easier to define restrictions on data
- It is easier to validate the correctness of data
- It is easier to convert data between different data types

XML Schemas use XML Syntax

Another great strength about XML Schemas is that they are written in XML:

- You don't have to learn a new language
- You can use your XML editor to edit your Schema files
- You can use your XML parser to parse your Schema files
- You can manipulate your Schemas with the XML DOM
- You can transform your Schemas with XSLT

XML Schema

The Schema above is interpreted like this:

- `<xs:element name="note">` defines the element called "note"
- `<xs:complexType>` the "note" element is a complex type
- `<xs:sequence>` the complex type is a sequence of elements
- `<xs:element name="to" type="xs:string">` the element "to" is of type string (text)
- `<xs:element name="from" type="xs:string">` the element "from" is of type string
- `<xs:element name="heading" type="xs:string">` the element "heading" is of type string
- `<xs:element name="body" type="xs:string">` the element "body" is of type string

XML Schema

XML Schema is an XML-based alternative to DTD:

```
<xs:element name="note">

  <xs:complexType>
    <xs:sequence>
      <xs:element name="to" type="xs:string"/>
      <xs:element name="from" type="xs:string"/>
      <xs:element name="heading" type="xs:string"/>
      <xs:element name="body" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>

</xs:element>
```

How to work XML Schema

Help:

- <https://support.microsoft.com/en-us/help/315533/how-to-validate-an-xml-document-by-using-dtd-xdr-or-xsd-in-visual-basi>
- <https://www.w3schools.com/xml/default.asp>

How to create XML Schema using Eclipse

[..\..\Reading material\XML and XMLS\videoplayback.mp4](#)

How to create XML DTD Schema

[..\..\Reading material\XML and XMLS\XML With DTD Schema.mp4](#)

Limitations of DTDs

DTDs describe only the “grammar” of the XML file, not the detailed structure and/or types

This grammatical description has some obvious shortcomings:

- we cannot express that a “length” element must contain a non-negative number (*constraints on the type of the value of an element or attribute*)
- **The “unit”** element should only be allowed when “**amount**” is present (*co-occurrence constraints*)
- the “**comment**” element should be allowed to appear anywhere (*schema flexibility*)
- There is no subtyping / inheritance (*reuse of definitions*)
- There are no composite keys (*referential integrity*)

Example Schema

```
<?xml version=„1.0“ ?>
<xsd:schema xmlns:xsd=„http://w3.org/2001/XMLSchema“>
  <xsd:element name=„book“ type=„BookType“/>
  <xsd:complexType name=„BookType“>
    <xsd:sequence>
      <xsd:element name=„title“ type=„xsd:string“/>
      <xsd:element name=„author“ type=„PersonType“
        minOccurs=„1“ maxOccurs=„unbounded“/>
      <xsd:complexType name=„PersonType“>
        <xsd:sequence> ... <xsd:sequence>
      </xsd:complexType>
    <xsd:element name=„publisher“ type=„xsd:anyType“/>
  </xsd:sequence>
</xsd:complexType>
</xsd:schema>
```

Example Schema

```
<?xml version=„1.0“ ?>
```

```
<xsd:schema  
  xmlns:xsd=„http://w3.org/2001/XMLSchema“>
```

...

```
</xsd:schema>
```

Schema in a separate XML Document

Vocabulary of Schema defined in special Namespace. Prefix „xsd“ is commonly used

There is a Schema for Schemas (don't worry!)

„schema“ Element is always the Root

Example Schema

```
<xsd:element name=„book“ type=„BookType“/>
```

„element“ Element in order to declare elements

„name“ defines the name of the element.

„type“ defines the type of the element

Declarations under „schema“ are **global**

Global element declarations are potential roots

Example: „book“ is the only global element,
root element of a valid document must be a „book“.

The type of a „book“ is BookType (defined next).

Example Schema

```
<xsd:complexType name=„BookType“>  
  <xsd:sequence>  
    ...  
  </xsd:sequence>  
</xsd:complexType>
```

User-defined complex type

Defines a sequence of sub-elements

Attribute „name“ specifies name of Type

This Typedefinition is **global**.
Type can be used in any other definition.

Example Schema

```
<xsd:sequence>  
  <xsd:element name=„title“ type=„xsd:string“/>  
</xsd:sequence>
```

Local element declaration within a complex type
(„title“ cannot be root element of documents)
„name“ and „type“ as before
„xsd:string“ is built-in type of XML Schema

Example Schema

```
<xsd:element name=„author“ type=„PersonType“  
  minOccurs=„1“ maxOccurs=„unbounded“/>
```

Local element declaration

„PersonType“ is user-defined type

„minOccurs“, „maxOccurs“ specify cardinality of „author“
Elements in „BookType“.

Default: minOccurs=1, maxOccurs=1

Example Schema

```
<xsd:complexType name=„PersonType“>
  <xsd:sequence>
    <xsd:element name=„first“ type=„xsd:string“/>
    <xsd:element name=„last“ type=„xsd:string“/>
  </xsd:sequence>
</xsd:complexType>
```

Local type definition

„PersonType“ may only be used inside the scope of the definition of BookType.

The same syntax as for BookType.

Example Schema

```
<xsd:element name=„publisher“ type=„xsd:anyType“/>
```

Local element declaration

Every book has exactly one „publisher“
minOccurs, **maxOccurs** by default 1

„anyType“ is built-in Type

„anyType“ allows any content

„anyType“ is default type. Equivalent definition:

```
<xsd:element name=„publisher“ />
```

Example Schema

```
<?xml version="1.0" ?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:element name="book" type="BookType"/>
  <xsd:complexType name="BookType">
    <xsd:sequence>
      <xsd:element name="title" type="xsd:string"/>
      <xsd:element name="author" type="PersonType"
        minOccurs="1" maxOccurs="unbounded"/>
      <xsd:complexType name="PersonType">
        <xsd:sequence> ... <xsd:sequence>
      </xsd:complexType>
    <xsd:element name="publisher" type="xsd:anyType"/>
  </xsd:sequence>
</xsd:complexType>
</xsd:schema>
```

Global vs. Local Declarations

Instances of global element declarations are potential root elements of documents

Global declarations can be referenced

```
<xsd:schema xmlns:xsd=„...“>
  <xsd:element name=„book“ type=„BookType“/>
  <xsd:element name=„comment“
    type=„xsd:string“/>
  <xsd:ComplexType name=„BookType“>
    ... <xsd:element ref=„comment“ minOccurs=„0“/>...
```

Constraints

- „ref“ not allowed in global declarations
- No „minOccurs“, „maxOccurs“ in global Decl.

Attribute Declarations

Attributes may only have a SimpleType

SimpleTypes are, e.g., „string“ (more later)

Attribute declarations can be global

- **Reuse declarations with ref**

Compatible to Attribute lists in DTDs

- Default values possible
- Required and optional attributes
- Fixed attributes
- (In addition, there are „prohibited“ attributes)

Attribute Declarations

```
<xsd:complexType name=„BookType“>
  <xsd:sequence> ... </xsd:sequence>
  <xsd:attribute name=„isbn“ type=„xsd:string“
    use=„required“ />
  <xsd:attribute name=„price“ type=„xsd:decimal“
    use=„optional“ />
  <xsd:attribute name=„curr“ type=„xsd:string“
    fixed=„EUR“ />
  <xsd:attribute name=„index“ type=„xsd:idrefs“
    default=„“ />
</xsd:complexType>
```

Anonymous Types

PersonType need not be „named“

```
<xsd:complexType name=„BookType“>
```

```
...
```

```
<xsd:element name=„author“>
```

```
  <xsd:complexType>
```

```
    <xsd:sequence>
```

```
      <xsd:element name=„first“ type=„xsd:string“/>
```

```
      <xsd:element name=„last“ type=„xsd:string“/>
```

```
    </xsd:sequence> </xsd:complexType>
```

```
</xsd:element> ...
```

Simple Elements + Attributes

```
<xsd:element name=„price“>
  <xsd:complexType>
    <xsd:simpleContent>
      <xsd:extension base=„xsd:decimal“ >
        <xsd:attribute name=„curr“ type=„xsd:string“/>
      </xsd:extension>
    </xsd:simpleContent>
  </xsd:complexType> </xsd:element>
```

Valid Instance:

```
<price curr=„USD“ >69.95</price>
```

Element + Attributes, no Content

```
<xsd:element name=„price“>
```

```
  <xsd:complexType>
```

```
    <xsd:attribute name=„curr“ type=„xsd:string“/>
```

```
    <xsd:attribute name=„val“ type=„xsd:decimal“/>
```

```
  </xsd:complexType>
```

```
</xsd:element>
```

Valid Instance:

```
<price curr=„USD“ val=„69.95“ />
```

Task To Do

The purpose of this practical is to use XSLT and XPath to display in the browser information contained in an XML document. We will modify an existing XSL file to add functionality; we will

use the browser XSL processor to perform the transformation between XML and HTML for display.

Task to Do

Linking the XML file with the XLS stylesheet and adding more columns

- Open the file [cd_catalog.xml](#) in your browser and check how it is displayed
- Open and explore in your text editor the xsl file provided: [cd_catalog.xsl](#)
- Using your text editor, now add the following line in the xml document:, just after the first line: `<?xml-stylesheet type='text/xsl' href='cd_catalog.xsl' ?>`
- Open again the file `cd_catalog.xml` in your browser (or refresh it if you already have it opened). Now it should display differently, as an HTML table with two columns, one for Title and one for Artist.
- Create a new style sheet `cd_catalog2.xsl`, based on the first one. This time you want to display the remaining data columns: COUNTRY, COMPANY, PRICE and YEAR.
- Change also the colour of the text background in the heading to “cyan”.
- Link the XML document with this new style sheet and display it in the browser.

Task To Do

Sorting

- Create a new stylesheet `sort.xls` based on your previous `cd_catalog2.xls`. This time you want to display the content sorted by YEAR.
- Link the XML document with this new style sheet and display it in the browser.

Conditional and sorting

- Create a new stylesheet `if+sort.xls` based on your previous `sort.xls`. This time you want to display only the CDs coming from the UK, again sorted by year.
- Link the XML document with this new style sheet and display it in the browser

Checkpoints	
<code>sort.xls</code>	Using the browser, the <code>cd_catalog.xml</code> file should display all the rows sorted by year.
<code>if+sort.xls</code>	Using the browser, the <code>cd_catalog.xml</code> file should display all the CDs from the UK sorted by year.

THE END

